Hiện tượng Bùng nổ Gradient (Exploding Gradients)



Nguyên nhân

Nếu giá trị riêng lớn nhất của Whh > 1, gradient bị nhân lên theo cấp số nhân.



Hậu quả

Phát nổ thành NaN hoặc vô cực.

Cập nhật trọng số quá lớn, làm mất ổn định hoàn toàn mạng, Loss dao động điên cuồng.



Giải pháp

Gradient Clipping

Cắt cụt gradient nếu nó vượt quá một ngưỡng nhất định trước khi cập nhật. Thường xuyên được áp dụng mặc định trong các Framework như Keras/PyTorch.

Gemini Notebook

Đặc trưng của Dữ liệu Chuỗi thời gian (Time Series)



Nhiễu trắng (White noise):

Các biến động ngẫu nhiên.



Tính thời vụ (Seasonality):

Các mẫu lặp lại đều đặn (Ngày, tuần, tháng).



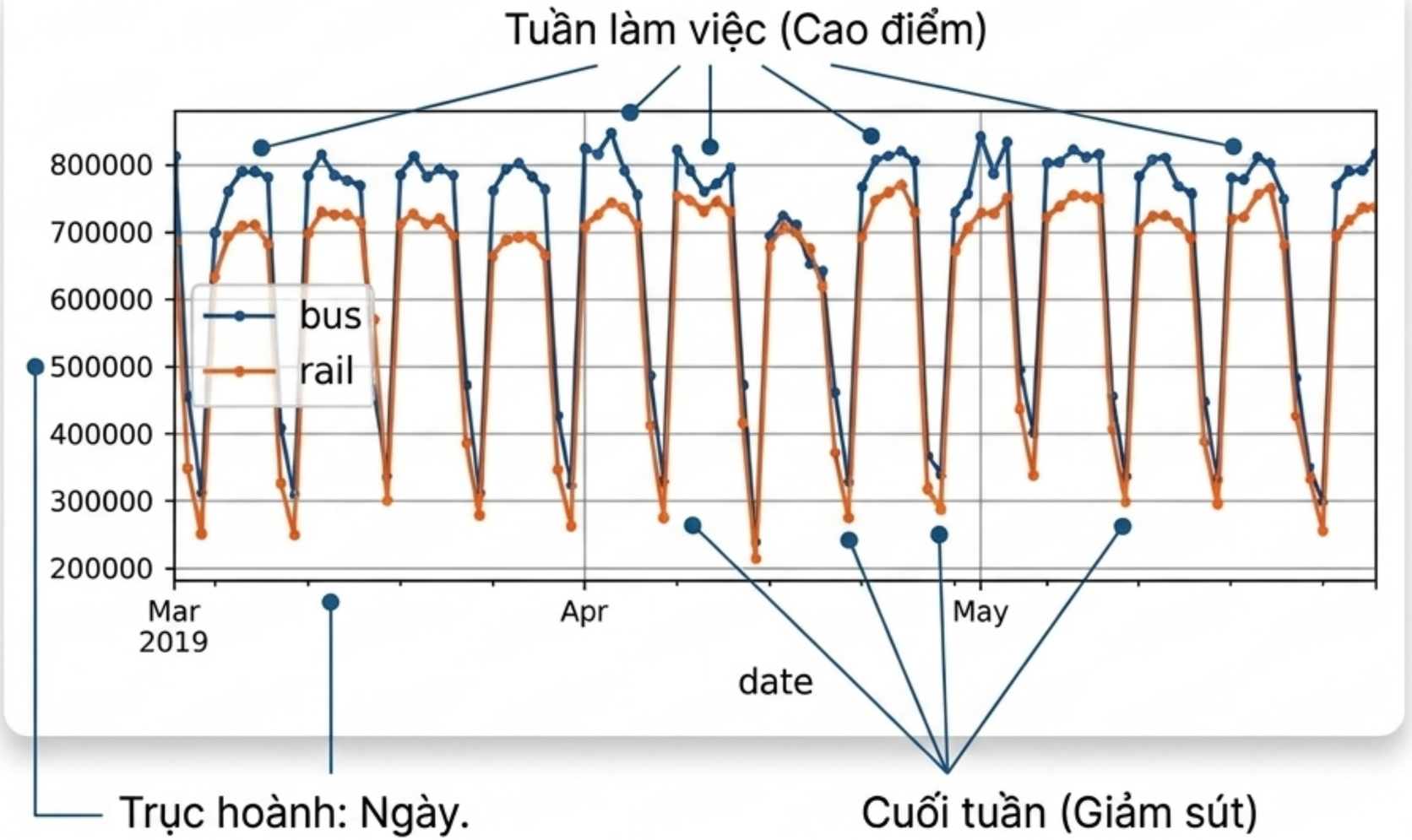
Xu hướng (Trend):

Tăng trưởng hoặc suy giảm chậm trong thời gian dài.

Tiền xử lý dữ liệu để loại bỏ những yếu tố này đôi khi hiệu quả hơn để mạng nơ-ron tự học từ đầu.

Gemini Notebook

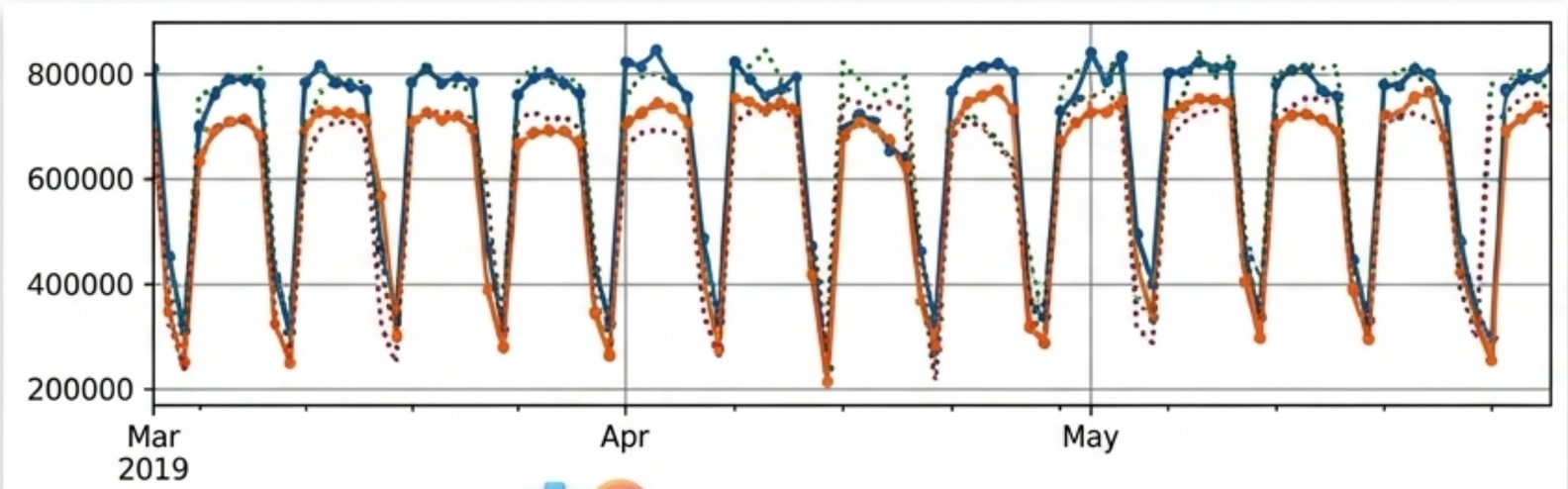
Phân tích Dữ liệu Thực tế: Hành khách Chicago (2019)



Biểu đồ cho thấy các **biến động liên tục và nhịp nhàng.** Mỗi **'khối' lởm chởm** tương ứng với **một tuần làm việc và cuối tuần.**

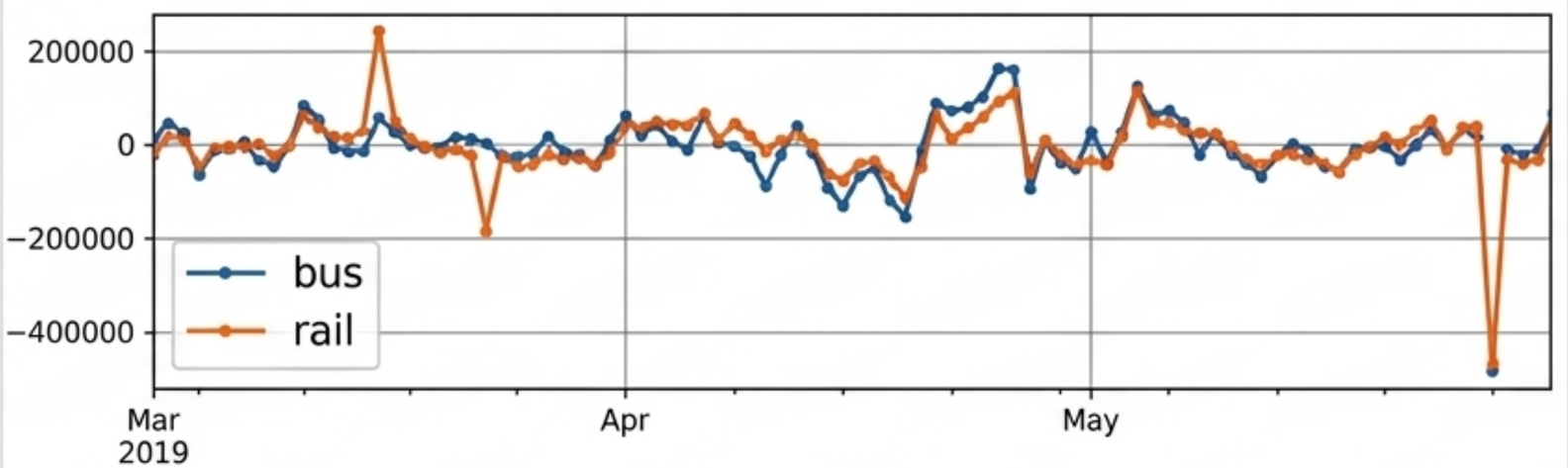
Gemini Notebook

Kỹ thuật Tiền xử lý: Khử tính Thời vụ Hàng tuần



Biểu đồ trên: Chuỗi gốc so với chuỗi bị dịch (trễ 7 ngày).

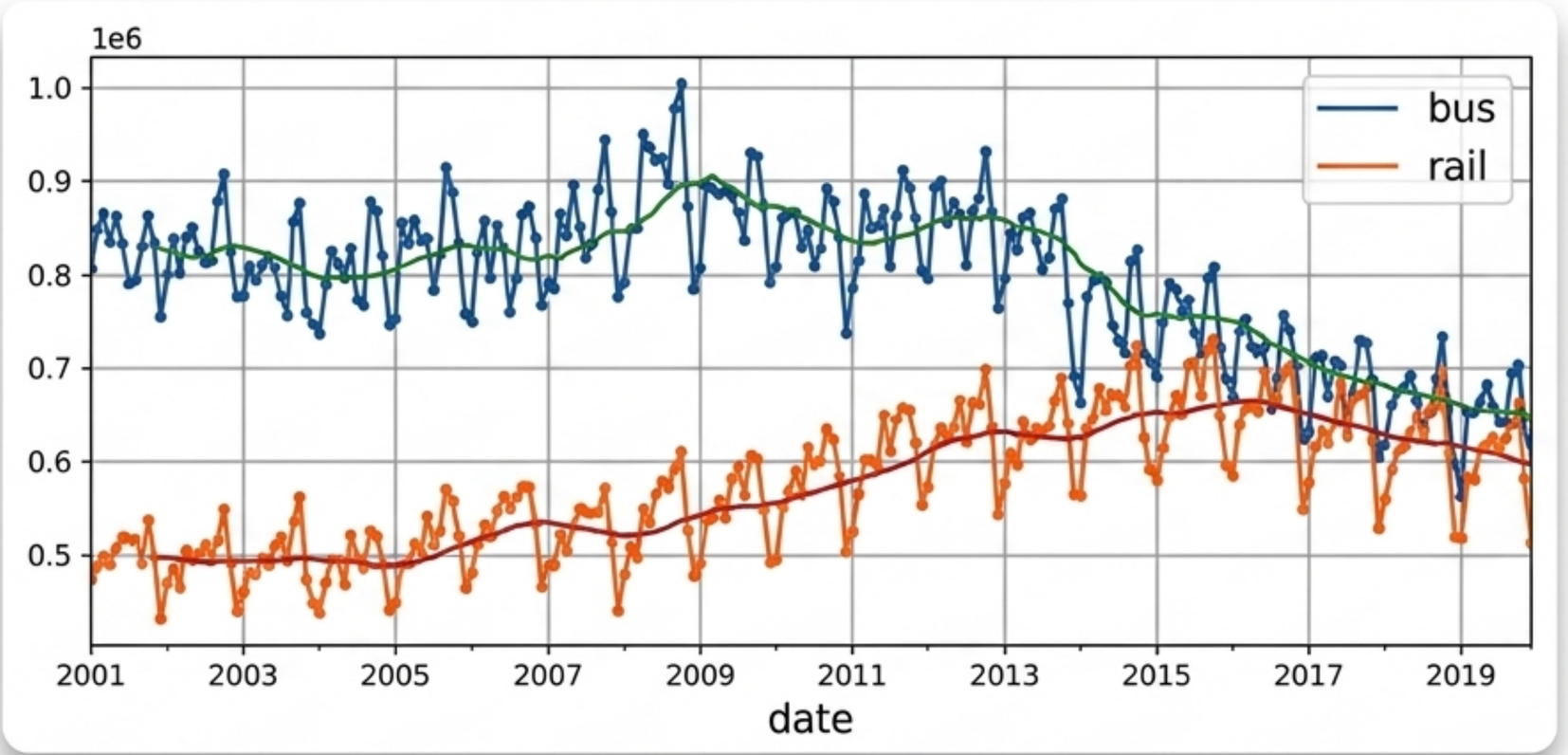
$y_t - y_{t-7}$ (Phép tính sai phân trễ 7 ngày).



Biểu đồ dưới: Chuỗi sai phân mượt mà, stationary (dừng). Đã loại bỏ được nhiễu do khác biệt giữa ngày thường và cuối tuần.

Gemini Notebook

Biến động Vĩ mô & Nhu cầu ‘Trí nhớ Dài hạn’



Mở rộng ra 15 năm (2001-2015), biểu đồ cho thấy rõ ràng các ngày lễ hội khiến lượng khách sụt giảm lớn hàng năm, và tổng thể hành khách biến động vĩ mô (Macro Trend).

Để RNN mô hình hóa được điều này, nó cần ‘Trí nhớ Dài Hạn’ vươn xa hàng trăm bước — điều này là cực kỳ khó nếu chỉ dùng SimpleRNN.

Gemini Notebook

Kỹ thuật Tiền xử lý: Sai phân 12 tháng (Dài hạn)



Áp dụng sai phân 12 tháng liên tục. (Hình 15-9).

Chuỗi kết quả cân bằng hơn quanh mốc 0.

Mạng RNN hiện đại có thể **tự phát hiện** các trend này. Tuy nhiên, trong thực tế, đưa vào chuỗi sai phân thường giúp mô hình **hội tụ nhanh hơn** và **dự đoán chính xác hơn** so với việc dùng dữ liệu gốc hỗn độn.

Gemini Notebook

Khởi tạo SimpleRNN với Keras

```
import tensorflow as tf
from tensorflow import keras

# Tao mang RNN 1 lop, 1 hidden unit
model = keras.Sequential([
    keras.layers.SimpleRNN(1, input_shape=[None, 1])
])

model.compile(loss="mse", optimizer="adam")
history = model.fit(X_train, y_train, epochs=20,
                    validation_data=(X_valid, y_valid))
```

API Huấn luyện mô phỏng nơ-ron hồi quy cơ bản.

Gemini Notebook

Giải mã Tham số Keras (Keras Parameters Explained)



```
SimpleRNN(1)
```

Tham số 1 là số lượng nơ-ron trạng thái ẩn (**dimensionality of output space**). Chọn 1 vì đây là dự đoán 1 bước tiếp theo (đại lượng vô hướng).



```
input_shape=[None, 1]
```

- **None**: Chiều dài chuỗi (Time steps). Keras cho phép chuỗi dài ngắn tùy ý.
- **1**: Số chiều (Features) tại mỗi bước thời gian (Chỉ có 1 đặc trưng: lượng hành khách).



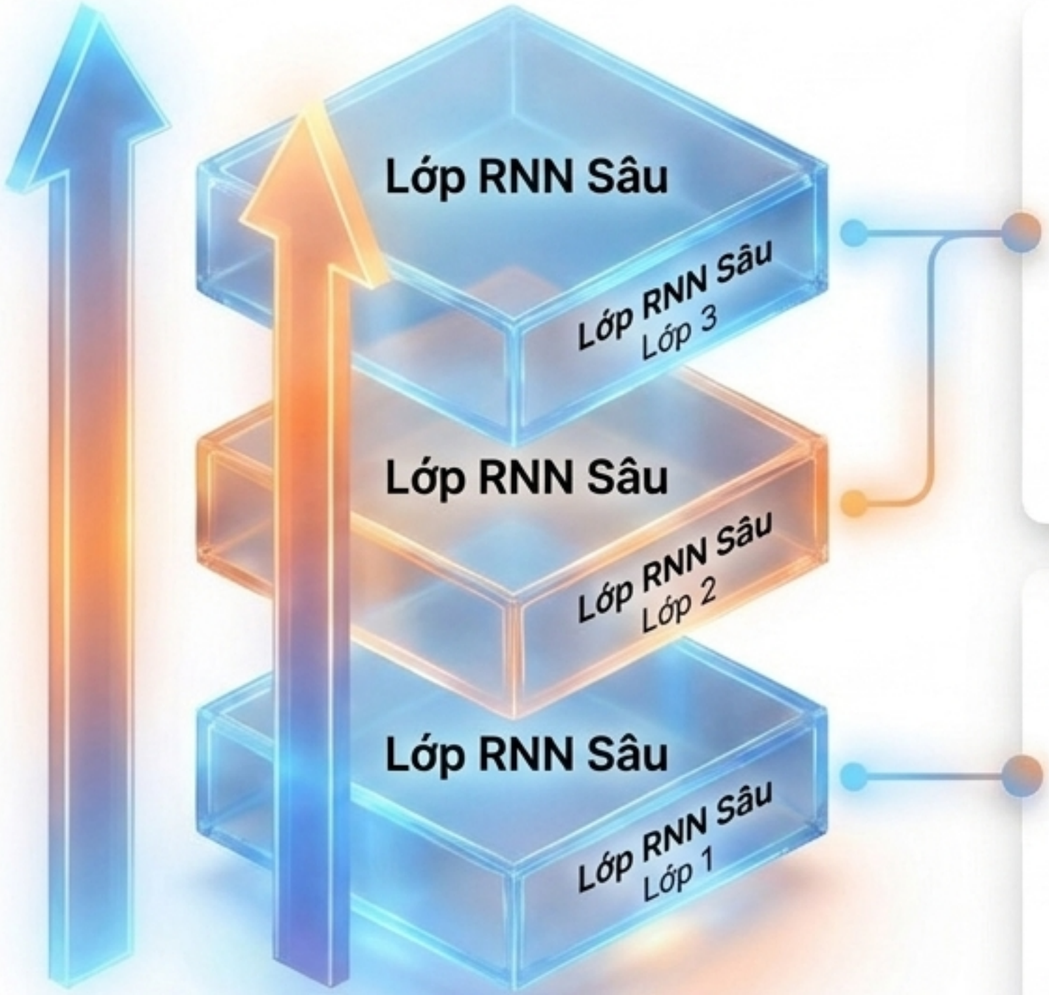
Hàm Loss

Thường dùng cho dự báo chuỗi là **mse** (Mean Squared Error) hoặc **mae** (Mean Absolute Error).

Gemini Notebook

Kiến trúc Deep RNN (Mạng RNN Sâu)

Tương tự MLP Sâu, ta xếp chồng (stacking) các lớp RNN lên nhau tạo thành Deep RNN.



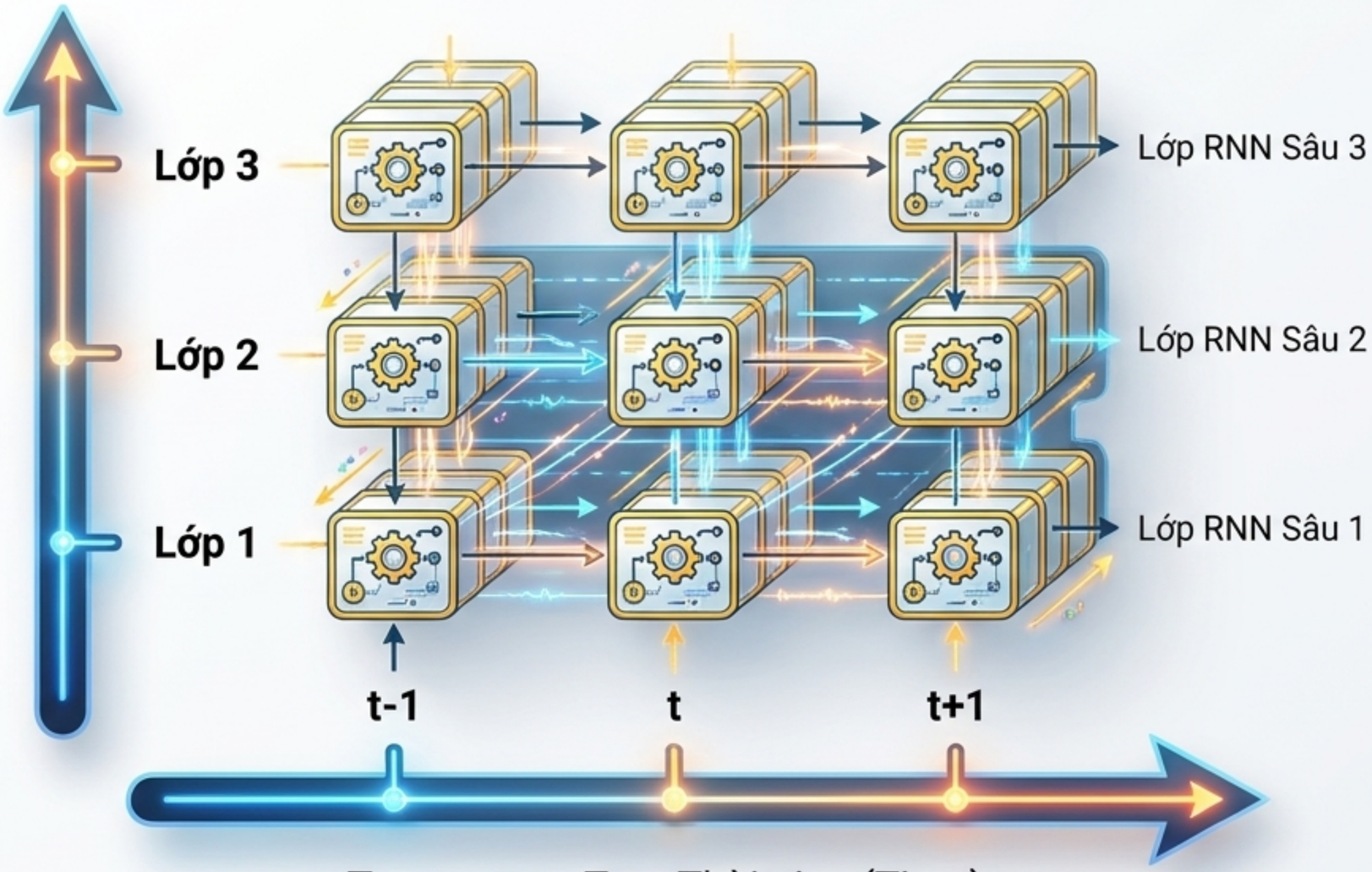
2. Đầu ra trạng thái ẩn **h(t)** của lớp dưới làm đầu vào cho lớp trên tại cùng thời điểm t.

1. Lớp dưới cùng nhận dữ liệu đầu vào.

Gemini Notebook

Sơ đồ Lan truyền trong Deep RNN

Trục dọc:
Trục Không gian / Chiều sâu (Space/Depth)



Trục ngang: Trục Thời gian (Time)

Gemini Notebook

Keras Code: Xây dựng Deep RNN



YÊU CẦU QUAN TRỌNG: Để lớp RNN dưới có thể đẩy chuỗi output lên lớp RNN trên, nó **BẮT BUỘC** phải bật **return_sequences=True**.

```
deep_model = keras.Sequential([
    # Lớp RNN 1: Trả về chuỗi cho lớp 2
    keras.layers.SimpleRNN(20, return_sequences=True,
        input_shape=[None, 1]),
    # Lớp RNN 2: Trả về chuỗi cho lớp cuối
    keras.layers.SimpleRNN(20, return_sequences=True),

    # Lớp RNN cuối: Chỉ trả về vector cuối cũng (Seq2Vec)
    keras.layers.SimpleRNN(1)
])
```

Gemini Notebook

Chiến lược Dự báo Đa bước (Multi-step Forecasting)

Dự đoán tuần tự (Autoregressive)

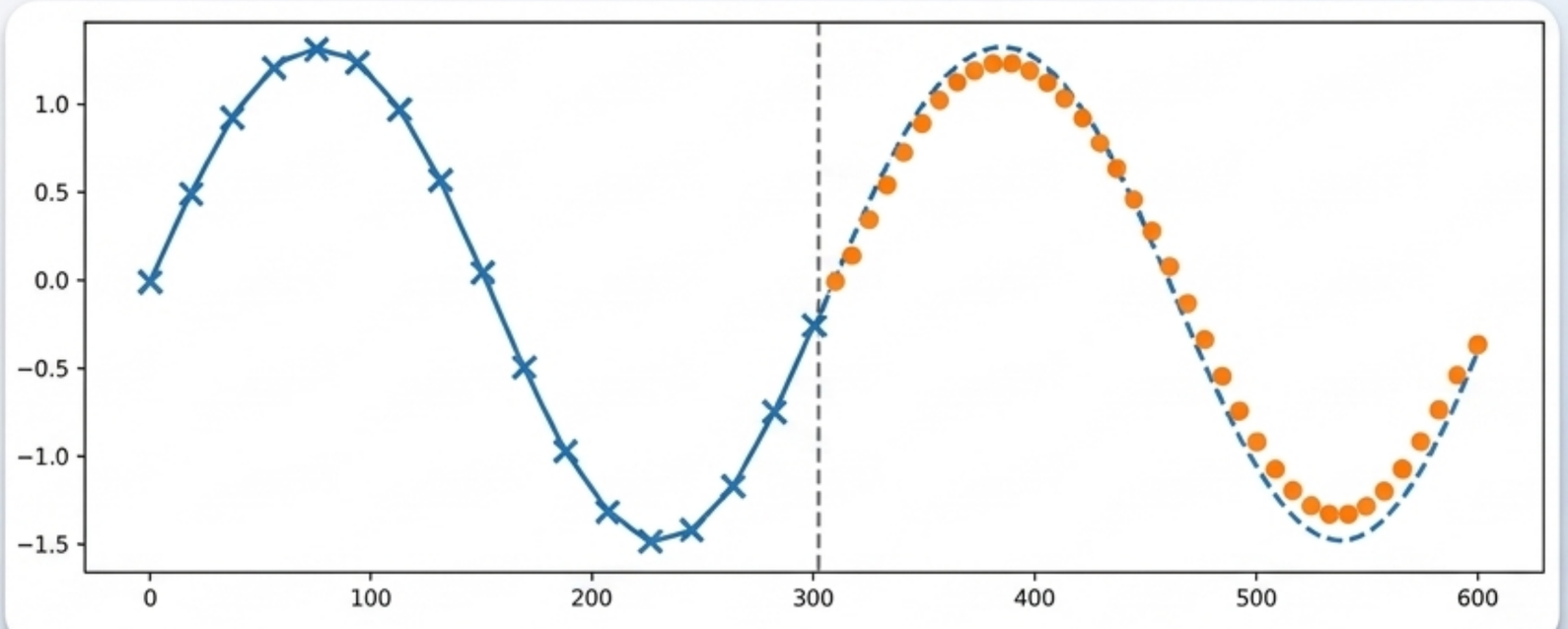
Dự đoán t+1, gộp vào mảng đầu vào để đoán t+2, lặp lại 10 lần.

(Nhược điểm: Dễ tích lũy sai số).

Dự đoán đồng thời

Đổi lớp cuối thành Dense(10) xuất 10 giá trị cùng lúc.

(**Ưu điểm**: Thường ổn định hơn).



Dấu × (Đầu vào)

Đường đứt nét (Target)

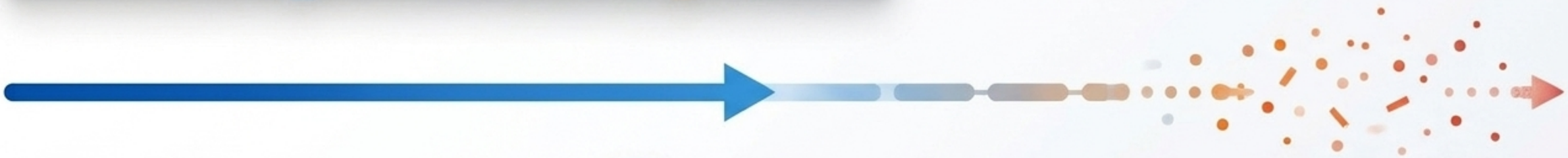
Dấu chấm tròn (Dự đoán của mạng)

Mạng bám sát rất tốt đường đi tự nhiên của sóng dốc.

Gemini Notebook

Hạn chế của SimpleRNN: Phụ thuộc xa (Long-Term Dependencies)

"Tôi sống ở Pháp từ nhỏ, sau 20 năm chuyển đến Mỹ, tôi vẫn nói trôi chảy tiếng [X]."



The Problem:

Mạng cần nhớ chữ “Pháp” ở quá khứ **rất xa** để đoán chữ “Pháp” ở hiện tại.

Vanishing Gradient:

Trọng số W nếu nhỏ thì Vanishing Gradient làm chữ “Pháp” bốc hơi sau **vài bước thời gian**. SimpleRNN rất tồi trong việc nắm bắt phụ thuộc xa.

Gemini Notebook

Sự ra đời của Tế bào LSTM (Long Short-Term Memory)

Phát minh bởi Hochreiter và Schmidhuber (1997) để tạo ra **"Trí nhớ Dài hạn"**.



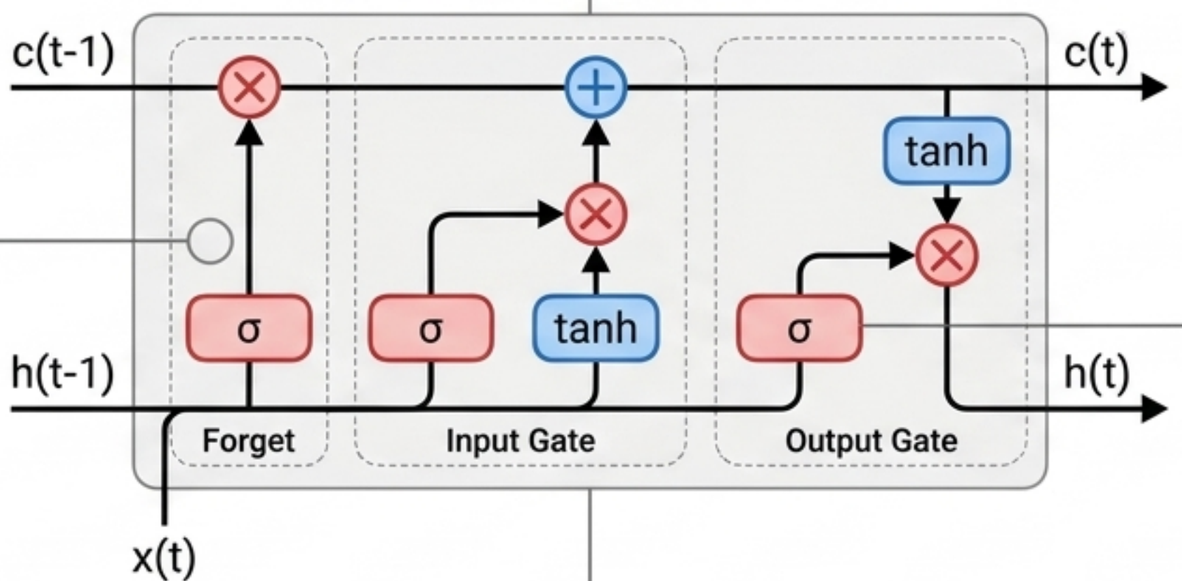
Thay vì chỉ truyền một Trạng thái ẩn h(t), LSTM **duy trì thêm một đường truyền phụ** gọi là **Trạng thái tế bào** (Cell State - c(t)).

c(t) giống như một đường ray cao tốc xuyên suốt, cho phép thông tin truyền đi rất xa mà không suy giảm. Các **Cổng (Gates)** kiểm soát việc ghi, đọc, xóa dữ liệu trên đường ray này.

Gemini Notebook

Giải phẫu Cấu trúc Tế bào LSTM

Đường ray ngang trên cùng:
Trạng thái tế bào c(t).



Legend

⊗, ⊕ là phép nhân phần tử và phép cộng vector.

Gate Valve Callout

Cổng (Gate) sử dụng hàm kích hoạt Sigmoid (σ) xuất giá trị [0, 1].

Hoạt động như một cái "van": 0 là đóng hoàn toàn, 1 là mở hoàn toàn.

Đường ray dưới:
Trạng thái ẩn h(t).

Gemini Notebook